

Apache Spark

From fundamentals to a production-like DORA pipeline

HDFS

YARN

SPARK

Concepts first

Architecture

DataFrames

DAG

Partitions

Shuffle

Performance

Then apply the concepts to DORA

Where Are We?

Trois responsabilités distinctes dans la plateforme

HDFS

Stockage distribué
Fichiers et blocs
Débit séquentiel élevé

YARN

Gestion des ressources
CPU, mémoire, queues
Planification

SPARK

Moteur de traitement
Transformations distribuées
Plans d'exécution

Spark traite les données. Il ne remplace ni le stockage HDFS ni la gestion de ressources YARN.

Why Spark?

Quand les pipelines deviennent trop complexes pour une succession de jobs simples



MapReduce reste un excellent modèle pédagogique, mais les chaînes de nombreux jobs deviennent lourdes à maintenir et à exécuter.

Pipelines

Plusieurs étapes et dépendances.

Iterative

Réutiliser les mêmes données plusieurs fois.

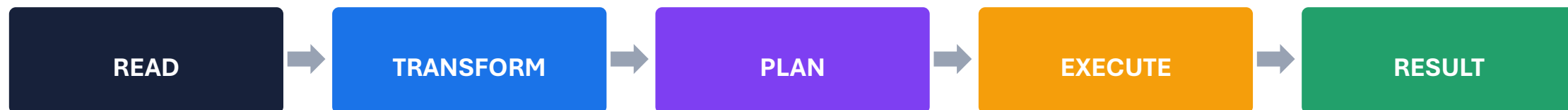
Expressive APIs

Décrire les transformations plus simplement.

Spark propose un modèle d'exécution plus général pour construire des pipelines distribués.

Spark in One Sentence

Un moteur distribué qui construit et exécute un plan de transformations



Spark sépare ce que le programme décrit de la manière dont le calcul sera réellement exécuté.

Spark Is Not Storage

Spark lit, transforme et écrit. Le stockage reste externe au moteur.

READ FROM

HDFS
Object Storage
JDBC
Files

PROCESS WITH

DataFrames
SQL
Transformations
Aggregations

WRITE TO

HDFS
Parquet / ORC
Object Storage
Databases

Une application Spark peut disparaître sans faire disparaître les données stockées dans HDFS ou S3.

What Can Spark Do?

Un moteur généraliste de traitement distribué

DataFrames

Données structurées et transformations déclaratives

Spark SQL

Requêtes SQL sur des données distribuées

Batch

Traitement de grands volumes par lots

Streaming

Traitement continu avec Structured Streaming

Machine Learning

Pipelines ML avec MLlib

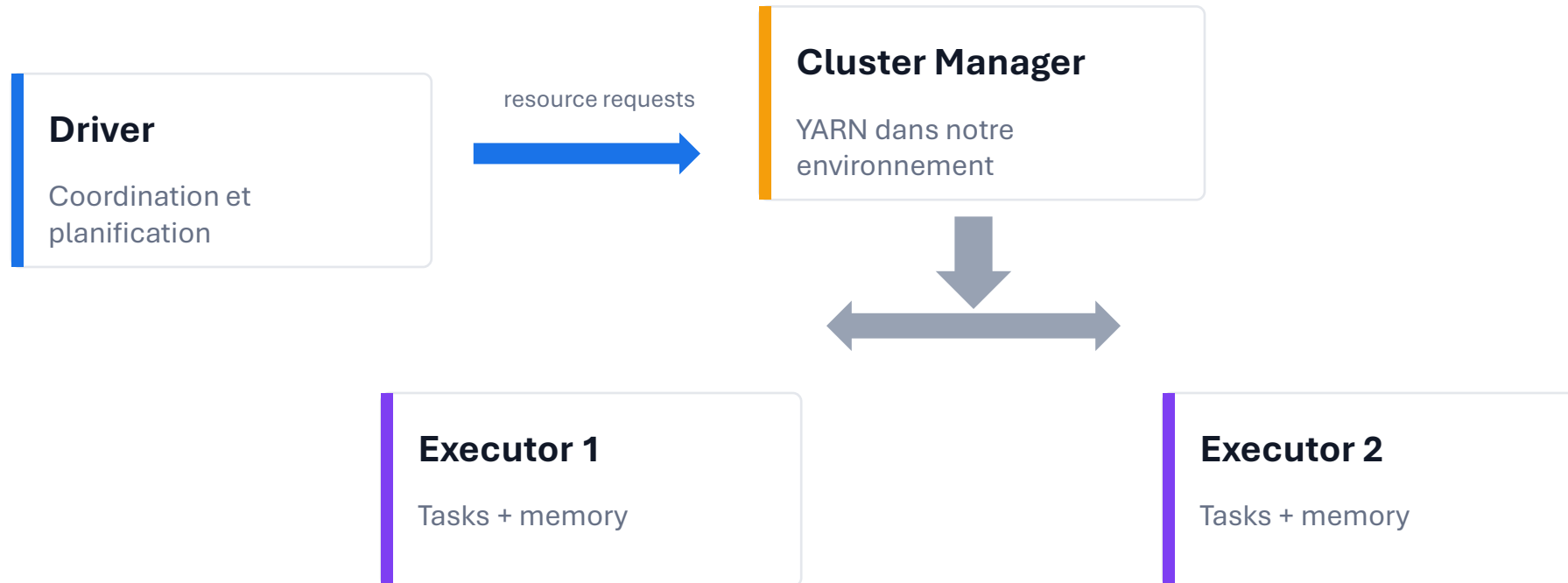
Graph Workloads

Traitements orientés graphe via bibliothèques adaptées

Dans ce module, nous nous concentrons surtout sur DataFrames, SQL, batch et performance.

Spark Architecture

Qui coordonne? Qui exécute? Qui fournit les ressources?



Le Driver coordonne. Le Cluster Manager alloue. Les Executors exécutent les Tasks.

The Driver

Le cerveau de l'application Spark

Driver responsibilities

- Exécuter le programme principal
- Créer la SparkSession
- Construire le plan
- Coordonner les Executors
- Suivre l'exécution

PySpark

```
spark = SparkSession.builder\n    .appName("MyApp")\n    .getOrCreate()
```

Le Driver n'est pas le lieu où toutes les données sont ramenées.

Le Driver coordonne le calcul distribué. Il ne doit pas devenir le goulot d'étranglement.

The Cluster Manager

Spark délègue l'allocation de ressources à un gestionnaire de cluster

YARN

Notre environnement pédagogique

Kubernetes

Conteneurs et orchestration

Standalone

Gestionnaire natif Spark

Le Cluster Manager fournit les ressources. Spark décide ensuite comment exécuter son plan avec ces ressources.

Executors

Les processus qui exécutent les Tasks

Executor 1

Task 1
Task 2
Cached partitions
Shuffle data

Executor 2

Task 3
Task 4
Cached partitions
Shuffle data

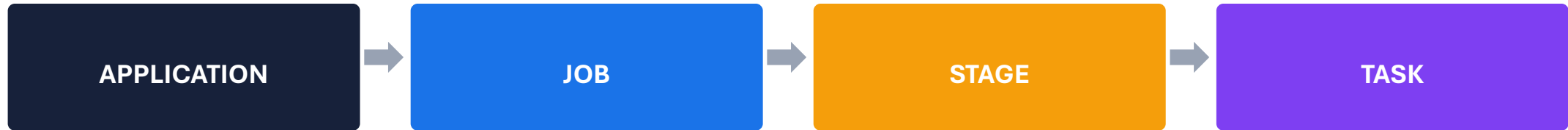
Executor 3

Task 5
Task 6
Cached partitions
Shuffle data

Les Executors exécutent les Tasks et peuvent conserver certaines partitions en mémoire ou sur disque.

Application - Job - Stage - Task

Les quatre niveaux à savoir distinguer



Application

Le programme Spark complet.

Job

Déclenché par une action.

Stage

Ensemble de Tasks sans nouvelle frontière de shuffle.

Task

Travail exécuté sur une partition.

Une action peut déclencher un Job, lui-même découpé en Stages puis en Tasks.

SparkSession

Le point d'entrée principal des APIs structurées

PySpark

```
from pyspark.sql import SparkSession
spark = SparkSession.builder
    .appName("SalesAnalytics")
    .getOrCreate()
```

SparkSession gives access to

- DataFrame API
- Spark SQL
- Catalog
- Read / Write APIs

Dans PySpark moderne, SparkSession est le point d'entrée principal pour les traitements structurés.

What Is a DataFrame?

Une collection distribuée organisée en colonnes nommées

timestamp	application	level	latency_ms
10:01	payment	INFO	120
10:02	auth	ERROR	850
10:03	payment	WARN	410

Why DataFrames?

- Schema connu
- Optimisations
- API déclarative
- Interopérabilité SQL

Un DataFrame décrit des données distribuées et leur structure.

DataFrame vs RDD

Deux niveaux d'abstraction

Dimension	RDD	DataFrame
Niveau	Bas niveau	Haut niveau
Structure	Objets distribués	Colonnes + schema
Optimisation	Moins d'informations	Plan optimisable
API	Fonctionnelle	Déclarative / SQL
Cours	Concept historique	API privilégiée

Pour les pipelines structurés, DataFrame est généralement l'API à privilégier.

DataFrame vs Dataset

Attention à la différence entre Python et JVM

	DataFrame	Dataset typé
Python	Oui	Pas d'API Dataset typée équivalente
Scala / Java	Oui	Oui
Typage	Schema de colonnes	Type JVM + encoders
Dans ce module	Principal modèle	Concept à connaître

En PySpark, nous travaillons essentiellement avec des DataFrames.

The Schema

Le contrat de structure des données

Schema

Root

```
-- app_id: string
-- status_code: integer
-- response_time_ms: integer
-- event_ts: timestamp
```

Why it matters

- Types explicites
- Détection d'erreurs
- Expressions adaptées
- Planification optimisée
- Contrat entre producteurs et consommateurs

Le schema donne à Spark une connaissance exploitable de la structure des données.

Explicit Schema vs inferSchema

Contrôle explicite ou inférence à partir des données

inferSchema

- Rapide pour explorer
- Dépend des données observées
- Peut ajouter une passe de lecture
- Moins contrôlé en production

Explicit schema

- Contrat stable
- Types attendus connus
- Erreurs plus prévisibles
- Meilleur contrôle du pipeline

Pour un pipeline industrialisé, un schema explicite est souvent préférable.

Transformations

Décrire une nouvelle version du dataset

Examples

```
adults = users
    .filter("age >= 18")
    .select("user_id", "country")
    .withColumn("is_eu", ...)
```

Typical transformations

```
select()
filter()
withColumn()
join()
groupBy()
```

Une transformation décrit un nouveau DataFrame mais ne demande pas encore un résultat final.

Actions

Demander un résultat et déclencher le calcul

show()

Afficher des lignes au Driver.

count()

Calculer un nombre.

write()

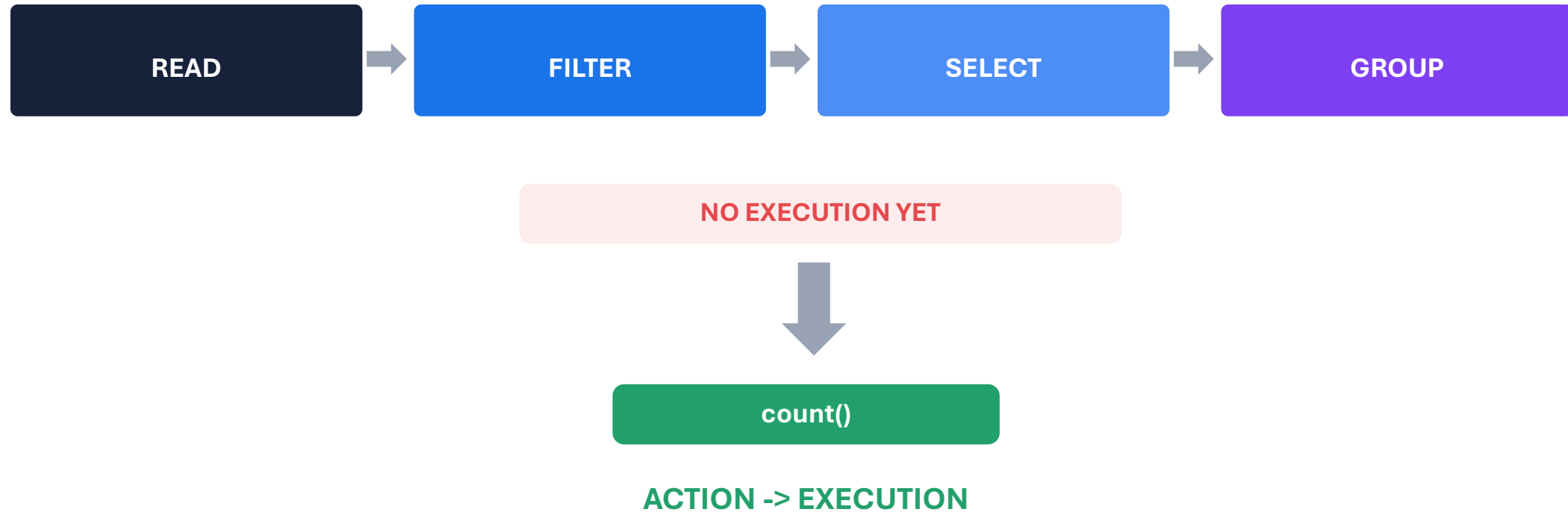
Matérialiser un résultat dans un stockage.

collect() est aussi une action, mais peut être dangereuse sur un gros dataset car elle rapatrie les données vers le Driver.

Transformation = description. Action = demande de résultat.

Lazy Evaluation

Spark attend une action avant d'exécuter le pipeline



Lazy evaluation permet à Spark d'analyser le pipeline avant de choisir comment l'exécuter.

Spark Builds a DAG

Les transformations forment un graphe de dépendances sans cycle



DAG = Directed Acyclic Graph

Le DAG décrit les dépendances entre transformations. Les Stages apparaissent ensuite lors de la planification physique.

Logical Plan

Ce que la requête signifie

User code

```
Sales  
.filter("country = 'FR'")  
.groupBy("product")  
.agg(F.sum("amount"))
```



Le plan logique décrit les opérations sans fixer encore chaque détail physique de leur exécution.

Physical Plan

Comment Spark choisit réellement d'exécuter la requête

Logical

Filter
Aggregate
Join

Optimizer

Catalyst rules
Statistics
Cost-based choices where available

Physical

Scan strategy
Join strategy
Exchange
Aggregation operator

Le code utilisateur ne décrit pas directement chaque opération physique exécutée.

explain()

Observer le plan plutôt que deviner

PySpark

```
df.explain(True)
```

What to look for

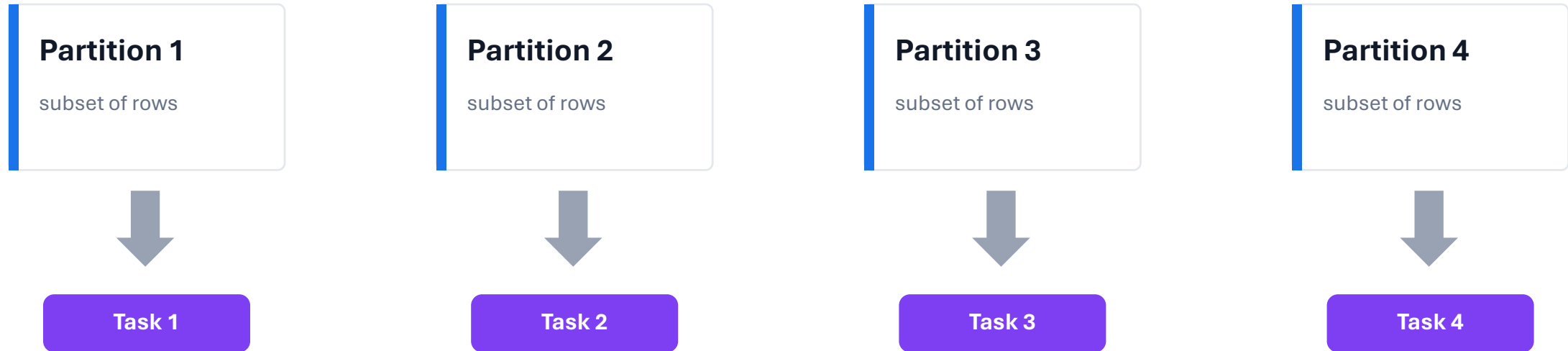
- Scan
- Filter
- Project
- Exchange
- Join strategy
- Aggregate

EXCHANGE est un signal important: il indique souvent une redistribution entre partitions.

Performance tuning commence par l'observation du plan.

Spark Partition

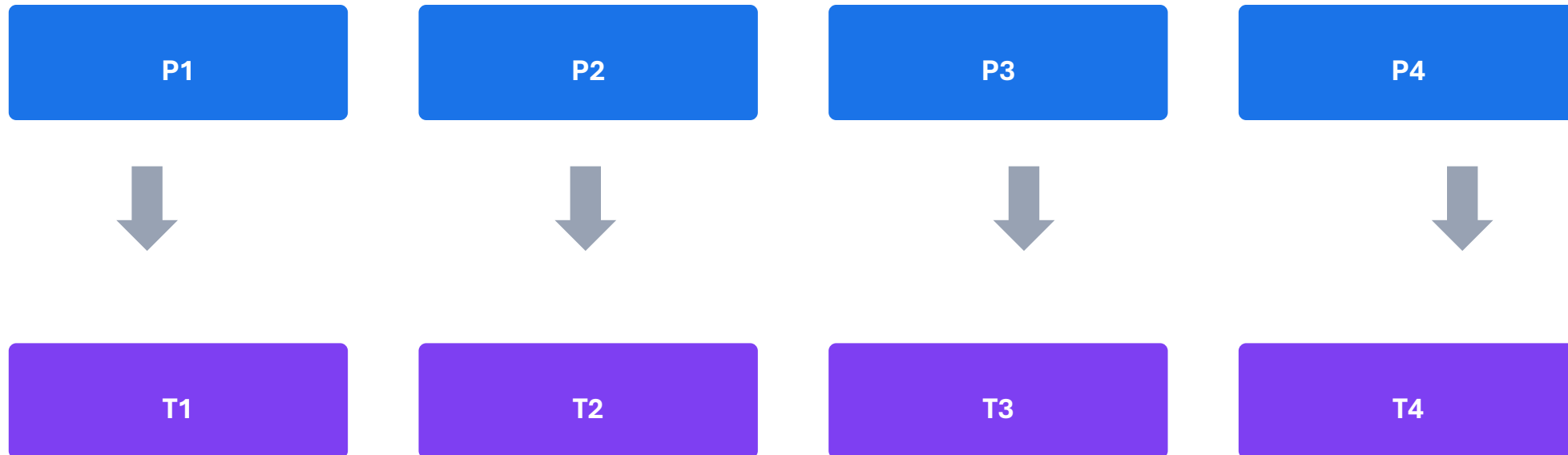
Une portion logique des données traitée par une Task



Une Task traite généralement une partition pour un Stage donné.

Partition -> Task

Le parallélisme est borné par le nombre de partitions disponibles



Plus de partitions peut augmenter le parallélisme, jusqu'aux limites des ressources et du coût de scheduling.

Why Partitions Matter?

Trop peu ou trop de partitions peuvent coûter cher

Too few

- Grosses partitions
- Faible parallélisme
- Risque de tâches longues
- Utilisation incomplète du cluster

Too many

- Très petites tâches
- Scheduling overhead
- Beaucoup de petits fichiers possibles
- Coût de coordination

Il n'existe pas de nombre universel de partitions. Il dépend du volume, des ressources et du traitement.

Narrow Transformations

Chaque partition de sortie dépend d'un nombre limité de partitions d'entrée

filter()

Garde ou retire des lignes dans chaque partition.

select()

Projette des colonnes sans redistribution globale.

withColumn()

Calcule une nouvelle colonne localement.

Les transformations narrow n'impliquent généralement pas de redistribution globale des données.

Wide Transformations

Certaines opérations nécessitent une redistribution entre partitions

groupBy()

Regrouper les mêmes clés.

join()

Rapprocher les lignes selon une clé.

distinct()

Comparer les valeurs globalement.

repartition()

Redistribuer explicitement.

Une wide transformation peut créer un shuffle et donc une nouvelle frontière de Stage.

What Is a Shuffle?

Redistribuer les données entre Executors selon de nouvelles partitions



Shuffle = déplacer des données pour que les lignes qui doivent être traitées ensemble se retrouvent dans les bonnes partitions.

Why Shuffle Is Expensive?

Plusieurs ressources sont sollicitées

Network

Transfert entre executors

Serialization

Conversion des données

Memory

Buffers et agrégations

Disk

Spill si la mémoire ne suffit pas

Sort

Organisation des partitions

Le shuffle n'est pas interdit. Il doit être compris et dimensionné.

Operations That Often Shuffle

Identifier les zones de vigilance dans un pipeline

groupBy

Regrouper par clé

join

Rapprocher deux datasets

distinct

Dédupliquer globalement

orderBy

Créer un ordre global

repartition

Changer la distribution

Observation

```
df.explain(True)    # look for Exchange
```

Repérer Exchange dans le plan est un bon réflexe pour comprendre les redistributions.

groupBy + agg

Passer des événements aux indicateurs

PySpark

```
result = df.groupBy("application").agg(  
    F.count("*").alias("events"),  
    F.avg("latency_ms").alias("avg_latency")  
)
```

What happens?

Les lignes sont regroupées par clé.
Une redistribution peut être nécessaire.
Les agrégats sont calculés par groupes.

Les agrégations globales ou par clé sont des opérations centrales mais potentiellement coûteuses.

Joins

Combiner deux datasets selon une clé

Join	Conserve à gauche	Conserve à droite	Usage
INNER	Correspondances	Correspondances	Seulement les lignes qui matchent
LEFT	Toutes	Correspondances	Préserver le dataset principal
RIGHT	Correspondances	Toutes	Préserver le dataset de droite
FULL	Toutes	Toutes	Vue complète des correspondances et écarts

Le choix du type de join est d’abord une décision fonctionnelle, puis une décision de performance.

Why LEFT JOIN?

Conserver les lignes du dataset principal même si la référence manque

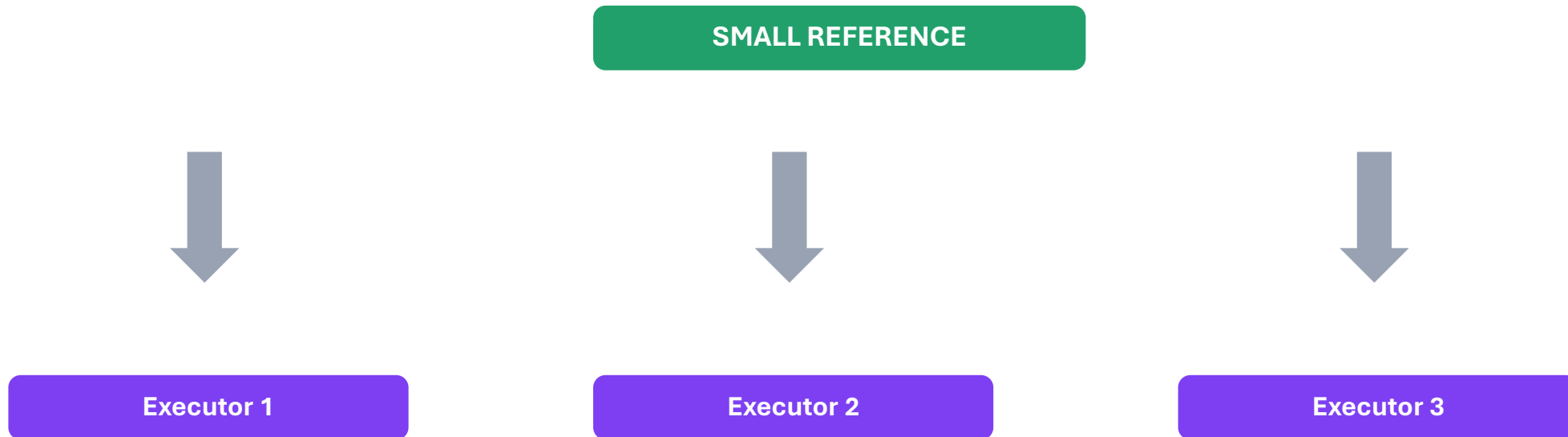


`customer_id = 999 -> reference not found -> transaction still preserved with NULL reference fields`

Un NULL après LEFT JOIN peut devenir un signal de qualité, pas seulement une erreur technique.

Broadcast Join

Envoyer un petit dataset à chaque Executor pour éviter de redistribuer le gros



Broadcast est efficace quand la table de référence est suffisamment petite pour être copiée sur les Executors.

When to Broadcast?

Une optimisation dépend toujours de la taille réelle

Good candidate

- Petit référentiel
- Réutilisé dans le join
- Tient confortablement en mémoire
- Évite le shuffle du gros dataset

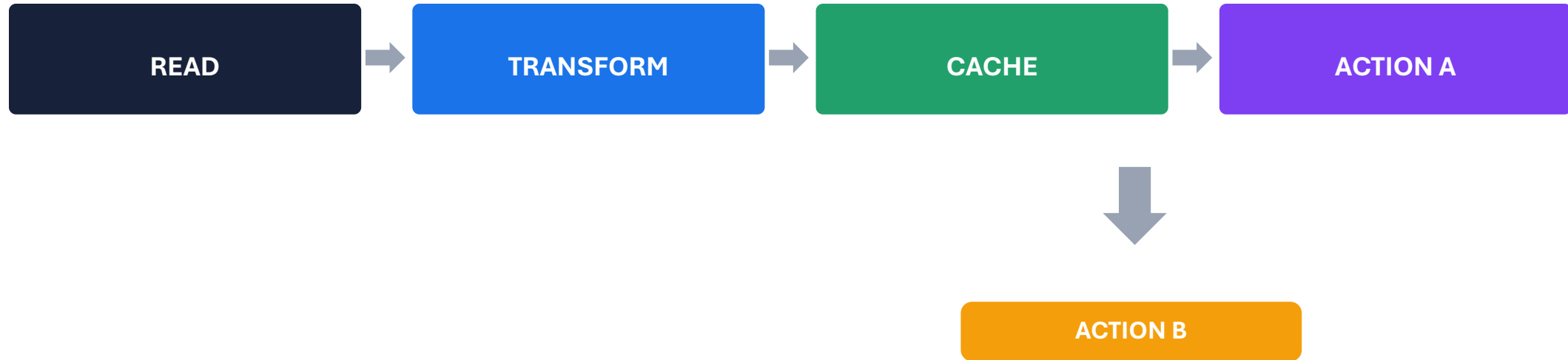
Bad candidate

- Dataset très volumineux
- Risque mémoire
- Temps de diffusion élevé
- Peut déstabiliser les Executors

Broadcast 5 MB peut être intéressant. Broadcast 50 GB ne l'est généralement pas.

cache() / persist()

Réutiliser un résultat intermédiaire sans recalculer toute sa lineage



Le cache est utile surtout quand un même dataset transformé est réutilisé plusieurs fois.

Spark Is Not "Everything in RAM"

La mémoire est une stratégie, pas une définition de Spark

Memory

Cache
Buffers
Aggregation state

Disk

Shuffle spill
Persist levels
Temporary files

Lineage

Spark peut recalculer certaines partitions perdues à partir des dépendances.

Spark utilise mémoire, disque, réseau et lineage selon les besoins du calcul.

repartition()

Changer explicitement le nombre ou la distribution des partitions



PySpark

```
df2 = df.repartition(8)
```

repartition() peut augmenter ou réduire le nombre de partitions mais implique généralement une redistribution.

coalesce()

Réduire le nombre de partitions avec moins de redistribution dans le cas courant



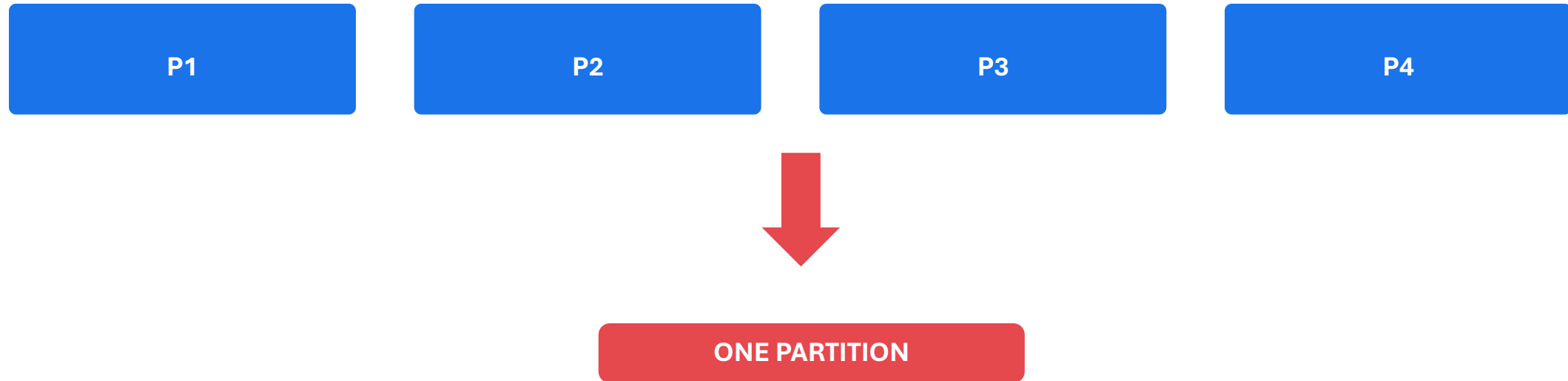
PySpark

```
df2 = df.coalesce(3)
```

coalesce() est surtout utilisé pour réduire le nombre de partitions sans redistribution complète systématique.

Why coalesce(1) Is Dangerous

Le désir d'un seul fichier peut détruire le parallélisme



Sur de gros volumes, une seule partition peut devenir un goulot d'étranglement majeur.

CSV vs Parquet

Format d'échange vs format analytique colonne

Dimension	CSV	Parquet
Structure	Texte ligne par ligne	Colonnaire
Schema	Externe / implicite	Métadonnées intégrées
Compression	Possible mais générique	Optimisée par colonnes
Lecture analytique	Souvent large	Column pruning possible
Usage	Échange / simplicité	Analytics / data lake

Pour les analyses répétées, Parquet est généralement plus adapté que CSV.

Why Columnar Formats?

Lire seulement les colonnes utiles

Query

```
SELECT app_id, latency_ms  
FROM events
```

CSV

app_id
level
user
latency
status
country
...
Read rows

Parquet

Read only
app_id
+
latency_ms
when the engine can prune
columns

Column pruning réduit le volume lu lorsque le format et le moteur le permettent.

partitionBy()

Organiser physiquement les fichiers de sortie selon une ou plusieurs colonnes

PySpark

```
df.write  
  .partitionBy("event_date")  
  .parquet("/data/events")
```

Physical layout

```
/data/events/  
  event_date=2026-01-15/  
  event_date=2026-01-16/  
  event_date=2026-01-17/
```

partitionBy() organise les fichiers. Ce n'est pas la même notion qu'une partition Spark en mémoire.

Spark Partition vs partitionBy

Même mot, deux concepts différents

Spark partition

Unité logique de données pendant l'exécution
Traitée par une Task
Influence le parallélisme

partitionBy()

Organisation physique des fichiers de sortie
Basée sur des valeurs de colonnes
Influence les lectures futures

Ne jamais utiliser ces deux sens de "partition" comme s'ils étaient interchangeables.

Partition Pruning

Éviter de lire les dossiers qui ne correspondent pas au filtre

2026-01-14

SKIP

2026-01-15

READ

2026-01-16

SKIP

Filter

```
df.filter(F.col("event_date") == "2026-01-15")
```

Un bon partitionnement physique permet de réduire les données parcourues pour certaines requêtes.

Spark UI

Observer une application pendant son exécution

Jobs

Actions et jobs

Stages

Frontières d'exécution

Tasks

Durées et distribution

Executors

CPU, mémoire, shuffle

Storage

Données persistées

La Spark UI est le premier outil pour comprendre ce que l'application fait réellement.

Spark History Server

Analyser les applications terminées à partir des event logs



Use after execution

Revoir jobs, stages, tasks, executors et métriques historiques.

History Server nécessite que les event logs Spark soient disponibles.

Diagnosing a Slow Job

Observer avant de modifier la configuration

Shuffle

Beaucoup de données transférées

Memory

Spill ou pression mémoire

Data skew

Quelques Tasks beaucoup plus longues

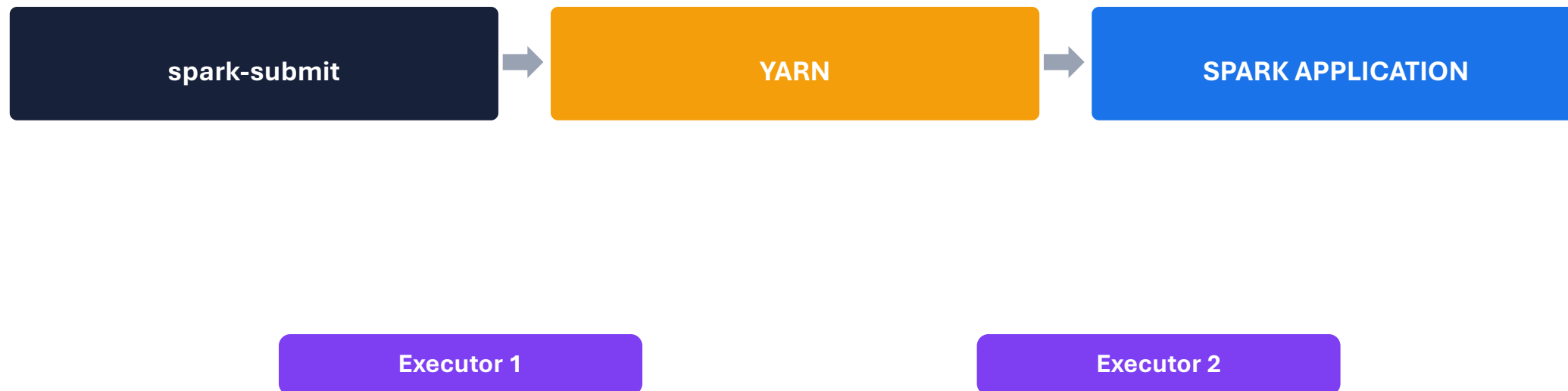
Partitions

Trop grandes ou trop nombreuses

Le tuning commence par des preuves: plan, métriques, distribution des Tasks et volume de shuffle.

Spark on YARN

Spark exécute ses Executors avec les ressources fournies par YARN



YARN fournit les ressources. Spark organise l'exécution de son application sur ces ressources.

Who Does What?

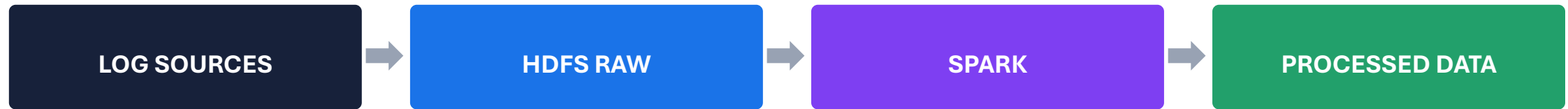
Une synthèse avant le cas d'usage

Element	Responsibility
HDFS	Stockage distribué des fichiers
YARN	Allocation CPU et mémoire
Driver	Coordination de l'application Spark
Executor	Exécution des Tasks
Spark	Planification et traitement distribué

Le rôle de chaque couche doit être clair avant de passer au projet fil rouge.

Back to the DORA Project

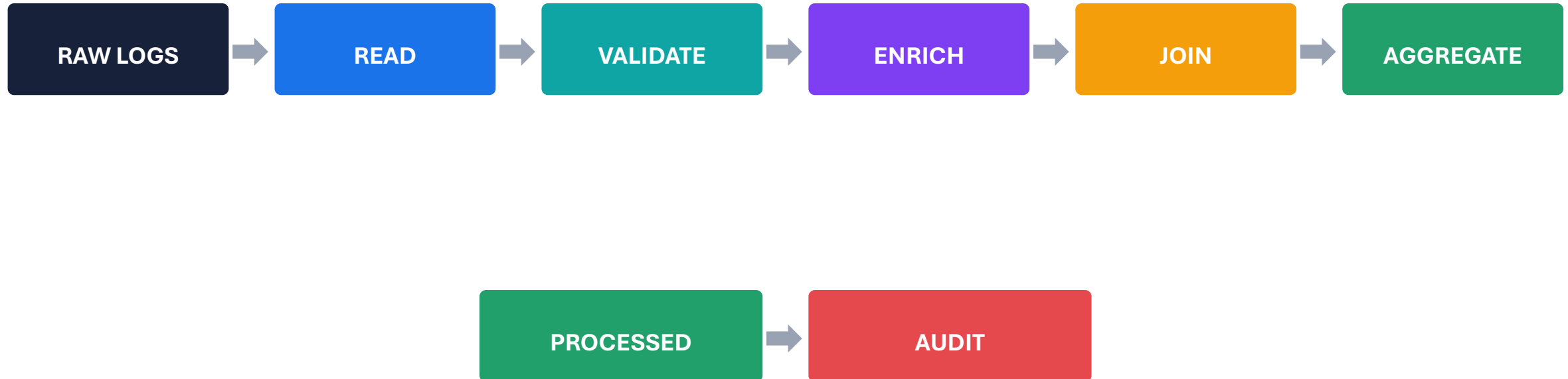
Spark devient le moteur de traitement du pipeline de logs



DORA fournit le contexte métier. Les concepts Spark restent ceux étudiés dans les slides précédentes.

DORA Pipeline

Réutiliser les briques Spark dans un pipeline complet



Chaque étape correspond à un concept Spark déjà vu: schema, transformations, joins, shuffle, agrégations et write.

Explicit Schema

Appliquer un contrat de données aux logs

Expected schema

event_ts	TIMESTAMP
app_id	STRING
level	STRING
status_code	INTEGER
response_time_ms	INTEGER

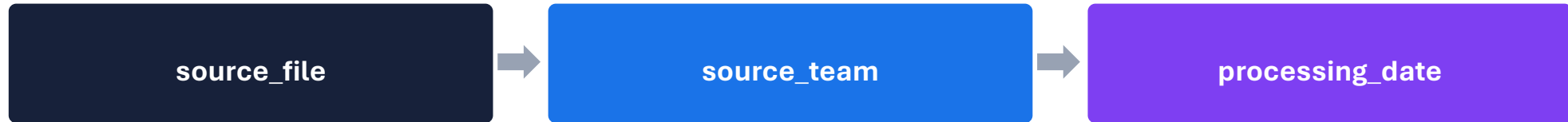
Why here?

- Détecter les écarts
- Garantir les types
- Stabiliser les transformations
- Rendre les règles reproductibles

Le schema devient un contrat de données entre producteurs de logs et pipeline Spark.

Data Lineage

Conserver l'origine et le contexte de traitement



Une métrique sans provenance est difficile à expliquer lors d'un contrôle ou d'un audit.

Lineage = pouvoir relier un résultat à ses données sources et à son exécution.

Enrichment

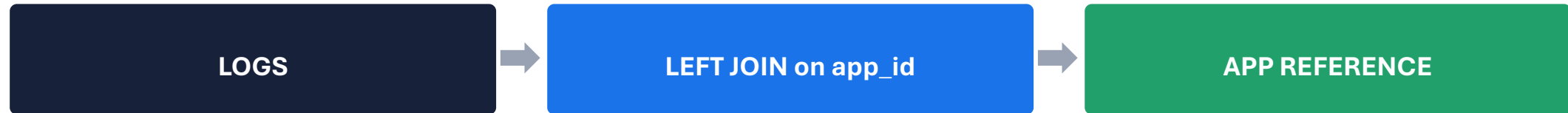
Transformer un événement technique en colonnes analytiques

Input	Derived column	Purpose
event_ts	event_date	Partition et analyse quotidienne
event_ts	event_hour	Analyse horaire
level	is_error	Comptage rapide des erreurs
level	is_warning	Comptage des warnings
response_time_ms	latency_bucket	Segmentation des latences

Les transformations Spark rendent les logs exploitables pour les agrégations suivantes.

Reference Join

Enrichir les logs avec un référentiel applicatif



Unknown app_id -> keep the log -> app_name = NULL -> quality signal

Le LEFT JOIN conserve la preuve technique même quand le référentiel est incomplet.

Broadcast Reference Data

Appliquer la stratégie de join déjà étudiée

PySpark

```
enriched = logs.join(  
    F.broadcast(apps),  
    "app_id", "left"  
)
```

Why broadcast?

Référentiel petit

Logs très volumineux

Éviter de redistribuer tous les logs

Réduire le shuffle

Broadcast est pertinent seulement si le référentiel tient confortablement en mémoire.

Metrics

Passer des événements aux indicateurs

event_count

Volume total

error_count

Nombre d'erreurs

warning_count

Nombre de warnings

error_rate

Taux d'erreur

avg_response_time

Latence moyenne

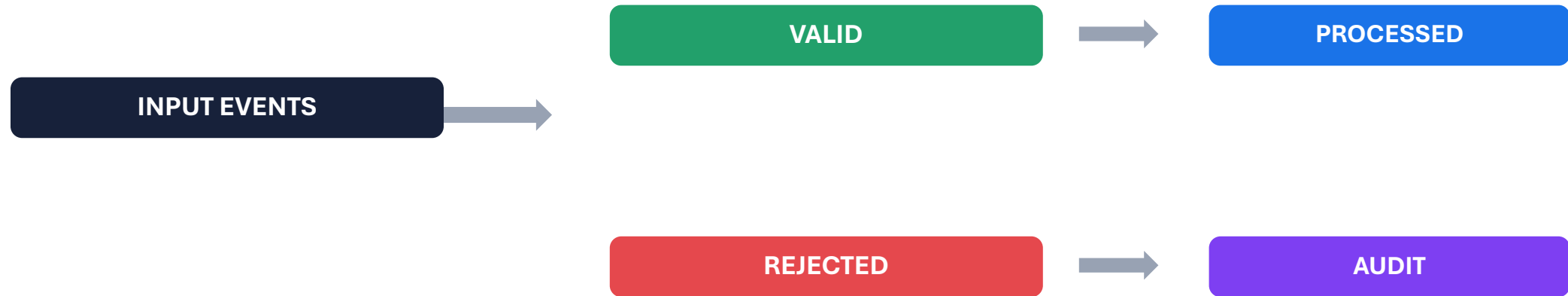
sla_breach_count

Dépassements SLA

Les agrégations transforment les logs détaillés en indicateurs exploitables et reproductibles.

Data Quality

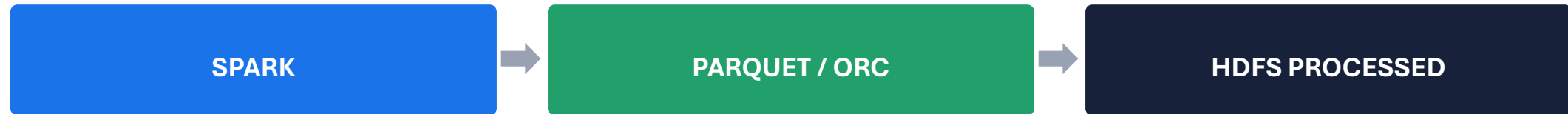
Séparer les données exploitables des données rejetées



Une donnée invalide ne doit pas disparaître silencieusement: conserver la raison et la provenance du rejet.

Persist the Results

Écrire des formats analytiques dans HDFS



Example

```
metrics.write.mode("overwrite").parquet("/datalake/processed/metrics")
```

Le format de sortie prépare les analyses Hive et les lectures ultérieures.

DORA Partition Strategy

Organiser les sorties sans recréer un small files problem

Physical layout

```
/processed/events/  
  event_date=2026-01-15/  
    app_id=payment-api/  
      app_id=auth-service/  
        event_date=2026-01-16/
```

Trade-offs

- Partition pruning
- Nombre de dossiers
- Nombre de fichiers
- Cardinalité de la clé
- Volume par partition

Question: partitionner par request_id serait-il une bonne idée?

Une bonne stratégie de partitionnement équilibre sélectivité des requêtes et taille raisonnable des partitions.

Observe the DORA Job

Utiliser explain(), Spark UI et History Server

explain(True)

Join strategy
Exchange
Scan
Filter
Aggregation

Spark UI

Jobs
Stages
Tasks
Shuffle
Executors

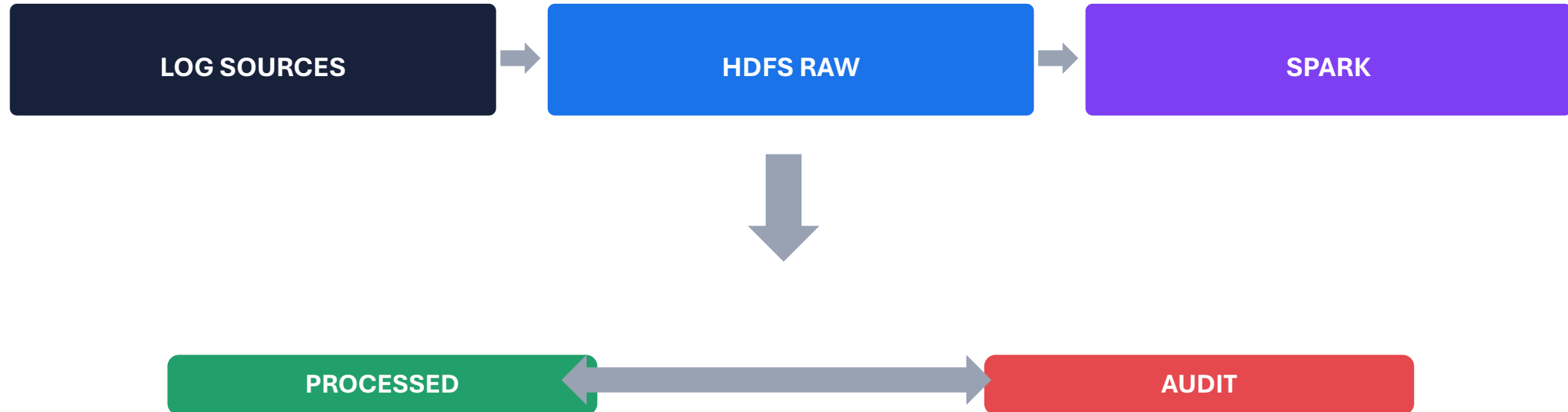
History Server

Rejouer l'analyse après la fin de l'application.

Le pipeline n'est pas terminé tant que nous ne savons pas expliquer son plan et ses coûts.

Complete DORA Architecture

Spark s'insère entre la zone RAW et les zones analytiques



Spark transforme. HDFS conserve les entrées et sorties. Hive exploitera ensuite les données structurées en SQL.

TP05 - What You Will Practice

Une mise en pratique des concepts Spark vus avant le projet

1 - READ

HDFS + schema

2 - TRANSFORM

filter + withColumn

3 - JOIN

reference + broadcast

4 - AGGREGATE

metrics + rates

5 - QUALITY

valid + rejected

6 - PERSIST

Parquet / ORC

7 - OBSERVE

explain + UI

8 - OPERATE

spark-submit + YARN

Objectif du TP: passer d'un code PySpark fonctionnel à un pipeline que l'on sait expliquer, mesurer et auditer.

Key Takeaways

Ce qu'un Data Engineer doit retenir de Spark

- 1 Driver coordonne, Executors exécutent
- 2 Transformations construisent un plan, actions déclenchent
- 3 Partitions déterminent le parallélisme des Tasks
- 4 Shuffle est un coût distribué central
- 5 DataFrames donnent à Spark un schema optimisable
- 6 Performance = plan + données + partitions + ressources
- 7 DORA est une application de ces concepts, pas leur définition

Comprendre Spark, c'est savoir relier le code au plan, le plan aux partitions et les partitions aux ressources.